

Instituto Superior de Engenharia de Lisboa

Departamento de Engenharia Informática

Licenciatura em Engenharia Informática e Multimédia

Computação na Nuvem

Trabalho Final

Ano Letivo:	2025/ 2026	
Docente:	José Simão	
Trabalho realizado por:	Diogo Santiago	N.º 51998
	Joana Ramos	N.º 50829
	Martim Ramos	N.º 51736
Data de Entrega:	26/05/2026	

Conteúdo

1. Introdução 1
2. Arquitetura do Sistema 2
   - 2.1. Visão Geral 2
   - 2.2. Componentes GCP 3
   - 2.3. Diagrama de Arquitetura 6
3. Contrato gRPC e Protobuf 6
   - 3.1. Serviços SF – Operações Funcionais 7
   - 3.2. Serviço SG – Gestão de Elasticidade 7
   - 3.3. Pressupostos Entre as Partes 8
4. Fluxo de Operações 9
   - 4.1. Lookup de Servidores gRPC 9
   - 4.2. Submissão de Imagem 10
   - 4.3. Processamento pela Labels App 11
   - 4.4. Consulta de Resultados 11
   - 4.5. Pesquisa por Características 11
   - 4.6. Logging – Cloud Function Pub/ Sub 12
5. Formatos de Dados e Mensagens 12
   - 5.1. Documentos Firestore 12
   - 5.2. Mensagem Pub/ Sub 13
   - 5.3. Blobs no Cloud Storage 13
6. Elasticidade 14
   - 6.1. Infraestrutura de Suporte 14
   - 6.2. Operações de Escalonamento 14
   - 6.3. Descoberta de Serviços 15
7. Aspetos Relevantes de Implementação 15
   - 7.1. Conflito de Versões do gRPC 15
   - 7.2. Dependências Transitivas no Cliente 16
   - 7.3. Ordem de Build dos Módulos Maven 16
   - 7.4. Autenticação nas Máquinas Virtuais 16
   - 7.5. Esgotamento de Quotas na Região da Bélgica 17
   - 7.6. Project ID na Cloud Function 17

7.7. Publicação Síncrona no Pub/ Sub 17
8. Pontos de Falha e Limitações 18
8.1. Processamento Assíncrono e Indisponibilidade de Resultados 18
8.2. Tolerância a Falhas no Pub/Sub 18
8.3. Ponto Único de Falha no Servidor gRPC 19
8.4. Ausência de Autenticação entre Componentes 19
8.5. Nomes Hardcoded na Configuração 19
8.6. Objetivos Atingidos e Não Atingidos 20
8.6.1. Objetivos Atingidos 20
8.6.2. Objetivos Não Atingidos 20
9. Conclusões 21

Índice de Ilustrações

Figura 1 - Diagrama de Arquitetura 6
Figura 2 - Diagrama de Sequência do Lookup 9

1. Introdução

O presente relatório descreve a arquitetura e implementação do sistema **CN2026Labels**, desenvolvido no âmbito da unidade curricular de Computação na Nuvem do curso de Engenharia Informática e de Computadores do Instituto Superior de Engenharia de Lisboa (ISEL), no ano letivo de 2025/2026.

O sistema tem como objetivo a submissão e processamento automático de imagens para deteção de características visuais (*labels*), como por exemplo *tree*, *street*, *night*, *cat* ou *fish*, com posterior tradução dessas características de inglês para português. Para além do processamento de imagens, o sistema foi desenhado com **elasticidade** como requisito fundamental, sendo capaz de aumentar ou diminuir dinamicamente a sua capacidade de processamento em função da carga de trabalho existente.

A solução assenta integralmente em serviços geridos da **Google Cloud Platform (GCP)**, nomeadamente *Cloud Storage, Cloud Pub/Sub, Cloud Firestore, Compute Engine, Cloud Run Functions, Cloud Vision API* e *Cloud Translation API*. A escolha de serviços geridos em detrimento de soluções auto-alojadas permite delegar na infraestrutura da Google as preocupações de disponibilidade, escalabilidade e manutenção, libertando a equipa de desenvolvimento para se focar na lógica aplicacional.

As funcionalidades do sistema são expostas a aplicações cliente através de uma **API gRPC**, composta por dois grupos de serviços distintos: o serviço **SF**, que agrupa as operações funcionais (submissão de imagens, consulta de resultados e pesquisa por características), e o serviço **SG**, responsável pelas operações de gestão da elasticidade do sistema. A escolha de gRPC em detrimento de uma API REST tradicional justifica-se pela necessidade de suportar *client streaming* no envio de imagens, pela eficiência na serialização de dados através de *Protocol Buffers*, e pela geração automática de código cliente e servidor a partir do contrato, reduzindo a probabilidade de erros de integração.

O contrato de comunicação entre os componentes foi definido em **Protocol Buffers (protobuf)**, garantindo uma interface fortemente tipada, eficiente e independente da linguagem. A implementação foi realizada em **Java**, utilizando **Maven** como sistema de gestão de dependências e build, organizada em três módulos independentes: o contrato gRPC (`grpcContract`), o servidor (`grpcServerApp`) e o cliente (`grpcClientApp`).

Este relatório encontra-se estruturado da seguinte forma: a secção 2 apresenta a arquitetura geral do sistema e os componentes envolvidos; a secção 3 detalha o contrato gRPC e os

pressupostos de comunicação entre as partes; a secção 4 descreve o fluxo de operações do sistema; a secção 5 documenta os formatos de dados e mensagens utilizados; a secção 6 aborda o mecanismo de elasticidade; a secção 7 discute os aspetos mais relevantes da implementação; e a secção 8 identifica os pontos de falha, limitações e objetivos atingidos.

2. Arquitetura do Sistema

Esta secção apresenta a arquitetura do sistema CN2026Labels, descrevendo os componentes que o constituem e a forma como interagem entre si. A arquitetura foi desenhada tendo em conta os requisitos de elasticidade, fiabilidade e desacoplamento entre componentes, tirando partido de um conjunto de serviços geridos da *Google Cloud Platform* que permitem delegar na infraestrutura da nuvem as preocupações de disponibilidade, escalabilidade e persistência de dados. Cada componente tem uma responsabilidade bem delimitada, comunicando com os restantes através de interfaces e protocolos adequados à natureza de cada interação.

2.1. Visão Geral

O sistema CN2026Labels segue uma arquitetura distribuída baseada em serviços da *Google Cloud Platform*, desenhada para suportar o processamento assíncrono e elástico de imagens. A solução é composta por vários componentes independentes que comunicam entre si através de mecanismos distintos consoante a natureza da interação: comunicação síncrona via gRPC entre o cliente e os servidores, e comunicação assíncrona via Pub/Sub entre os servidores gRPC e as instâncias de processamento de imagens.

Esta separação de responsabilidades permite que o sistema escale de forma independente em cada camada, aumentando ou diminuindo o número de instâncias de servidores gRPC e de instâncias da Labels App conforme a carga de trabalho, sem que isso afete o funcionamento global do sistema. Uma alternativa a esta separação seria ter um único componente a fazer tanto a receção dos pedidos como o processamento das imagens, mas tal abordagem eliminaria a possibilidade de escalar as duas camadas de forma independente e tornaria o sistema mais frágil, pois uma falha no processamento de imagens afetaria diretamente a receção de novos pedidos.

2.2. Componentes GCP

Cloud Storage

O **Cloud Storage** é utilizado como repositório persistente das imagens submetidas pelos clientes. Cada imagem é armazenada como um *blob* num *bucket* dedicado ao projeto, sendo-lhe atribuído um nome único que serve de referência nas fases subsequentes do processamento. O *bucket* foi configurado na região europe-west1 (Bélgica), numa configuração de armazenamento detalhado (*detailed*). A escolha desta região prendeu-se com a necessidade de alinhar geograficamente os serviços de armazenamento e base de dados, como se explica mais adiante. Uma alternativa ao *Cloud Storage* seria armazenar as imagens diretamente no Firestore como dados binários, mas o Firestore não é otimizado para ficheiros de grande dimensão, tornando o *Cloud Storage* a escolha natural para este tipo de conteúdo.

Cloud Pub / Sub

O **Pub/Sub** é o mecanismo de comunicação assíncrona entre os servidores gRPC e as instâncias da *Labels App*. Após a receção e armazenamento de uma imagem, o servidor gRPC publica uma mensagem num tópico dedicado, contendo o identificador do pedido, o nome do *bucket* e o nome do *blob*. As instâncias da *Labels App* partilham uma subscrição sobre esse tópico, seguindo o padrão *work-queue*, em que cada mensagem é entregue a apenas um *worker*, garantindo que o processamento não é duplicado.

A utilização do Pub/Sub em detrimento de uma chamada direta do servidor gRPC à *Labels App* é uma decisão de arquitetura fundamental. Uma chamada direta criaria um acoplamento forte entre os dois componentes, eliminando a possibilidade de ter múltiplos *workers* a processar imagens em paralelo e tornando o sistema mais vulnerável a falhas, se a *Labels App* estivesse indisponível no momento da chamada, o pedido seria perdido. O Pub/Sub garante que as mensagens ficam retidas até serem processadas, independentemente da disponibilidade momentânea dos *workers*.

Cloud Firestore

O **Firestore** é a base de dados do sistema, responsável por guardar toda a informação relevante sobre os pedidos de processamento. Opera em modo *Native*, que oferece maior flexibilidade na modelação de documentos e suporte a atualizações em tempo

real, em comparação com o modo *Datastore*, que é mais limitado e existe principalmente por razões de compatibilidade com sistemas legados.

Foi configurado na mesma região europe-west1 (Bélgica) que o *Cloud Storage*, de forma a minimizar a latência nas operações de leitura e escrita realizadas pela *Labels* App, que acede aos dois serviços em sequência durante o processamento de cada imagem. Colocar os dois serviços em regiões diferentes introduziria latência adicional e potenciais custos de transferência de dados entre regiões.

A escolha do Firestore em detrimento de uma base de dados relacional como o *Cloud SQL* justifica-se pela natureza dos dados armazenados. Cada pedido de processamento tem um conjunto variável de características detetadas, o que se adapta melhor a um modelo de documentos do que a um esquema relacional fixo. Para além disso, o Firestore é um serviço totalmente gerido, eliminando a necessidade de gerir instâncias de base de dados.

Compute Engine

O **Compute Engine** aloja as máquinas virtuais onde correm os dois tipos de componentes computacionais do sistema: os servidores gRPC e as instâncias da *Labels App*. Ambos são organizados em *instance groups*, o que permite gerir a sua escala de forma programática através das operações de gestão do serviço SG, aumentando ou diminuindo o número de instâncias em resposta à carga de trabalho.

A escolha do Compute Engine com *instance groups* em detrimento de um serviço de contentores como o *Cloud Run* justifica-se pelo controlo mais granular que oferece sobre as instâncias, nomeadamente a capacidade de gerir programaticamente o número de VMs através da API do Compute *Engine*, o que é um requisito explícito das operações de elasticidade do sistema.

Cloud Run Functions

São utilizadas duas **Cloud Functions** no sistema. A primeira, com trigger HTTP, implementa o serviço de *lookup* que devolve ao cliente a lista de endereços IP dos servidores gRPC ativos no *instance group*, permitindo que o cliente escolha a que se vai ligar. A implementação desta lógica como uma Cloud Function, em vez de a incorporar num servidor dedicado, elimina a necessidade de gerir mais uma instância e aproveita o modelo *serverless* para uma operação que é invocada de forma esporádica.

A segunda Cloud Function, com trigger Pub/Sub, recebe as mensagens publicadas pelo servidor gRPC e regista os pedidos de processamento no Firestore, funcionando como um mecanismo de logging desacoplado. A sua implementação é opcional, uma vez que o próprio worker da Labels App também escreve no Firestore após o processamento.

Cloud Vision API

A **Vision API** é invocada pela *Labels App* para analisar cada imagem e identificar as suas características visuais. Para cada característica detetada, a API devolve o nome em inglês e uma pontuação de confiança com valor entre 0 e 1, que representa o grau de certeza da deteção. A utilização de um serviço gerido de visão computacional elimina a necessidade de treinar ou manter modelos de *machine learning* próprios, o que estaria fora do âmbito deste projeto.

Cloud Translation API

A **Translation API** é também invocada pela *Labels App*, após a deteção das características, para traduzir cada *label* de inglês para português. A API recebe o texto a traduzir, o idioma de origem e o idioma de destino, identificados pelas siglas normalizadas, por exemplo "`en`" para inglês e "`pt`" para português. Uma alternativa seria manter uma tabela de tradução estática, mas esta abordagem seria limitada ao conjunto de *labels* previamente conhecidas, não cobrindo todas as características que a *Vision API* pode devolver.

A ***Translation API*** é também invocada pela *Labels App*, após a deteção das características, para traduzir cada *label* de inglês para português. A API recebe o texto a traduzir, o idioma de origem e o idioma de destino, identificados pelas siglas normalizadas.

2.3. Diagrama de Arquitetura

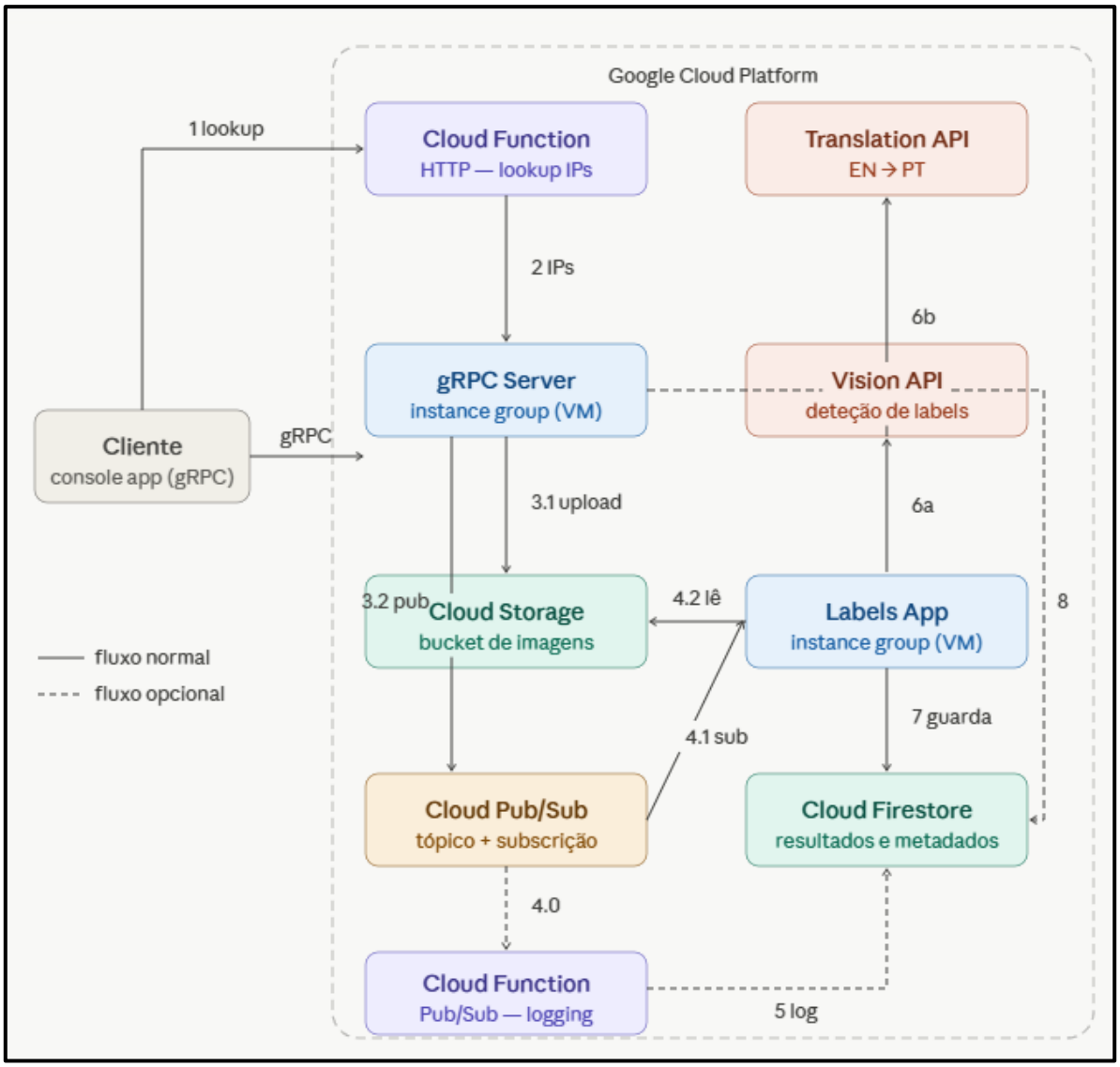


Figura 1 - Diagrama de Arquitetura

3. Contrato gRPC e Protobuf

Esta secção apresenta o contrato de comunicação definido entre o cliente e os servidores gRPC do sistema CN2026Labels, detalhando os serviços disponíveis, as mensagens trocadas e os pressupostos assumidos entre as partes envolvidas.

3.1. Serviços SF – Operações Funcionais

O serviço `FuncionalidadesGerais` agrupa as operações funcionais do sistema, disponibilizando três métodos de comunicação com o servidor gRPC.

O método `SubmeterImagem` permite ao cliente enviar uma imagem para processamento. A transferência é feita em modo cliente *streaming*, ou seja, o cliente envia o conteúdo do ficheiro dividido em blocos sucessivos do tipo `BlocoImagem`, cada um contendo o nome do ficheiro e um fragmento dos seus bytes. Esta abordagem permite enviar ficheiros de grande dimensão sem limitações de tamanho de mensagem, ao contrário de uma chamada unitária onde o ficheiro teria de ser enviado de uma só vez. No final da transmissão, o servidor responde com uma mensagem `RespostaSubmissao` que contém o identificador único do pedido, gerado no momento da receção, que o cliente deverá guardar para consultas futuras.

O método `ObterDetalhesImagem` permite ao cliente consultar os resultados do processamento de uma imagem previamente submetida. O cliente envia uma mensagem `PedidoDetalhes` com o identificador do pedido e o servidor responde com uma mensagem `RespostaDetalhes` contendo a data em que a imagem foi processada e a lista de características detetadas. Cada característica é representada por uma mensagem `DetalheCaracteristica`, que inclui o nome em inglês, a tradução em português e a pontuação de confiança atribuída pela Vision API, com o valor entre `0` e `1`.

O método `ProcuraPorCaracteristicas` permite pesquisar todas as imagens armazenadas no sistema que contenham uma determinada característica que tenham sido processadas dentro de um intervalo de datas. O cliente envia uma mensagem `PedidoPesquisa` com o nome da característica a pesquisar e as datas de início e fim do intervalo. O servidor responde com uma mensagem `RespostaPesquisa` contendo a lista dos nomes dos ficheiros que satisfazem os critérios indicados.

3.2. Serviço SG – Gestão de Elasticidade

O serviço `GestaoElasticidade` agrupa as operações de gestão do sistema, disponibilizando dois métodos que permitem ajustar dinamicamente a capacidade de processamento.

O método `EscalarServidoresGrpc` permite aumentar ou diminuir o número de instâncias de servidor gRPC em execução no *instance group* correspondente. O método `EscalarLabelsApp` tem o mesmo comportamento, mas aplicado ao `instance`

`group` das instâncias da *Labels App*. Ambos os métodos recebem uma mensagem PedidoEscala com o número desejado de instâncias e respondem com uma mensagem `RespostaEscala` que indica se a operação foi bem sucedida e uma mensagem descritiva do resultado.

A decisão de expor as operações de elasticidade através do mesmo contrato gRPC que as operações funcionais, em vez de as implementar numa interface de administração separada, simplifica a arquitetura do cliente, que só precisa de conhecer um único ponto de entrada para todas as operações do sistema.

3.3. Pressupostos Entre as Partes

A definição do contrato assenta num conjunto de pressupostos assumidos entre o cliente e o servidor que importa documentar.

Relativamente à submissão de imagens, o cliente é responsável por dividir o ficheiro em blocos de tamanho adequado antes de iniciar o *stream*, e por incluir o nome do ficheiro original no primeiro bloco enviado. O servidor assume que todos os blocos de um mesmo stream pertencem ao mesmo ficheiro e que o *stream* termina quando o cliente fechar a ligação.

Relativamente às datas, tanto no campo `data_processamento` da resposta de detalhes como nos campos `data_inicio` e `data_fim` da pesquisa, foi adotado o formato textual `YYYY-MM-DD`, de acordo com a norma ISSO 8601. Uma alternativa seria utilizar o tipo `Timestamp` do protobuf, que representa instantes no tempo com maior precisão, mas para os requisitos deste sistema o formato de data simples é suficiente e mais legível.

Relativamente ao identificador do pedido, o servidor garante que é único para cada submissão. O cliente deve preservá-lo após a submissão, pois é o único meio de consultar os resultados do processamento. Não existe nenhum mecanismo de recuperação do identificador caso este seja pedido pelo cliente.

Relativamente à disponibilidade dos resultados, o processamento das imagens é assíncrono. Tal significa que, após a submissão, os resultados podem não estar imediatamente disponíveis quando o cliente invocar o método `ObterDetalhesImagem`. Neste caso, o servidor devolve uma resposta com a lista de características vazia, cabendo ao cliente repetir a consulta após um intervalo de tempo.

4. Fluxo de Operações

Esta secção descreve, de forma detalhada, a sequência de operações que ocorrem no sistema CN2026Labels para cada uma das funcionalidades disponíveis. Os números de sequência utilizados correspondem aos apresentados no diagrama de arquitetura da secção **2.3 - DIAGRAMA DE ARQUITETURA**.

4.1. Lookup de Servidores gRPC

Antes de qualquer operação, o cliente necessita de conhecer o endereço IP de um servidor gRPC disponível. Para tal, invoca a *Cloud Function* com t*rigger* HTTP **(passo 1)**, que consulta o *Compute Engine* para obter a lista de endereços IP das instâncias ativas no *instance group* dos servidores gRPC **(passo 2)** e devolve essa lista ao cliente **(passo 3)**.

O cliente escolhe um dos endereços da lista **(passo 4)** e estabelece a ligação gRPC **(passo 5)**. Caso a ligação falhe, o cliente tenta outro endereço da lista ou repete o processo de *lookup* para obter uma lista atualizada. Este mecanismo garante que o cliente consegue sempre estabelecer ligação mesmo que uma ou mais instâncias estejam indisponíveis.

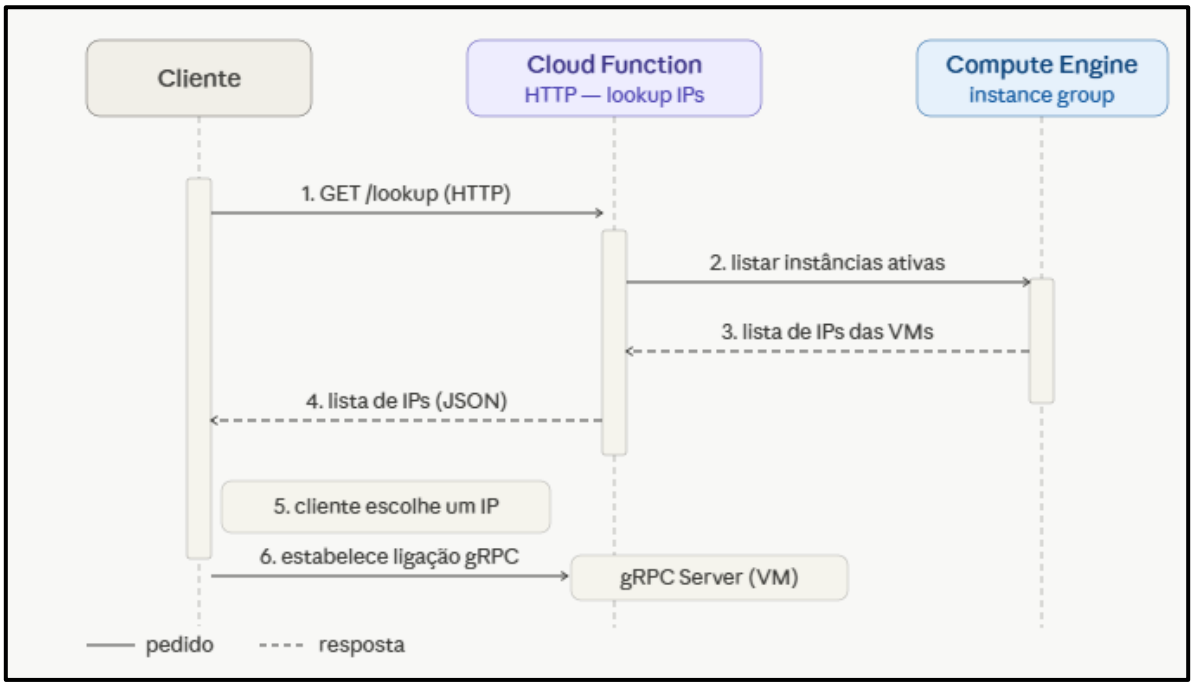


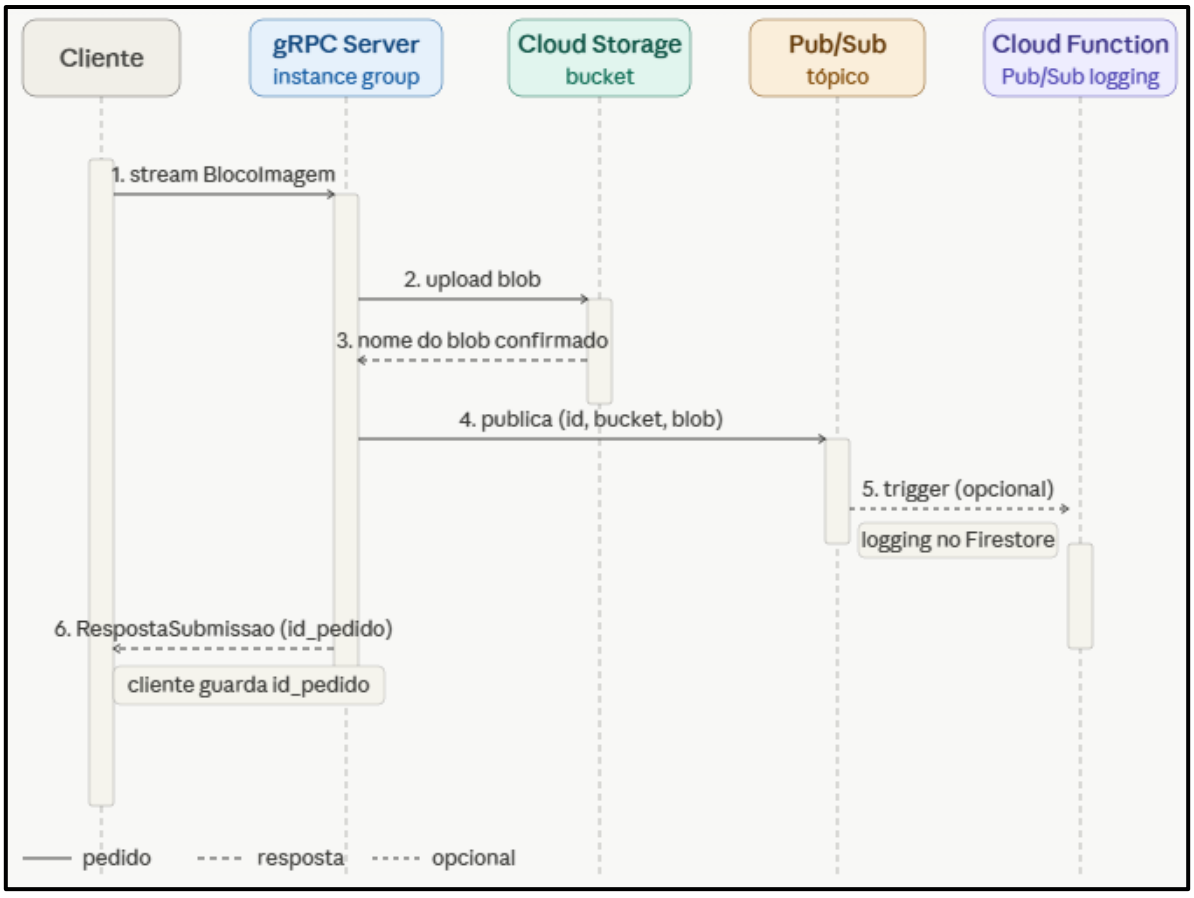
Figura 2 - Diagrama de Sequência do Lookup

4.2. Submissão de Imagem

Após estabelecer ligação com um servidor gRPC, o cliente invoca o método `SubmeterImagem`, enviando o conteúdo do ficheiro de imagem em *stream* de blocos sucessivos **(passo 1)**. Cada bloco contém o nome do ficheiro e um fragmento dos seus bytes em formato binário.

Do lado do servidor, à medida que os blocos vão chegando, o conteúdo é reconstituído e, no final do stream, a imagem é armazenada como um *blob* no *bucket* do *Cloud Storage* **(passo 2)**. O servidor recebe a confirmação do nome do *blob* **(passo 3)**, gera um identificador único para o pedido e publica uma mensagem no tópico Pub/Sub **(passo 4)** contendo o identificador do pedido, o nome do *bucket* e o nome do *blob*. Por fim, o servidor responde ao cliente com o identificador único gerado **(passo 6)**, que o cliente deverá guardar para consultas futuras.

Em paralelo e de forma opcional, a *Cloud Function* com *trigger* Pub/Sub recebe a mensagem publicada **(passo 5)** e regista o pedido de processamento no Firestore, funcionando como um mecanismo de *logging* desacoplado.



4.3. Processamento pela Labels App

A *Labels App* funciona omo um *worker* assíncrono que consome mensagens da subscrição do tópico Pub/ Sub seguindo o padrão *work-queue*. Cada mensagem é entregue a apenad uma instância da *Labels App*, garantindo que o processamento e cada imagem não é duplicado.

Ao receber uma mensagem, a *Labels App* extrai o nome do bucket e do blob e obtém a imagem correspondente no *Cloud Storage*. De seguida, invoca a *Cloud Vision API* para identificar as características visuais presentes na imagem, recebendo para cada uma o nome em inglês e a pontuação de cofiança. Para cada característica detetada, invoca a *Cloud Translation API* para obter a tradução para português.

Após o processamento completo, a Labels App guarda no Firestore toda a informação relevante do pedido: o identificador do pedido, o nome do ficheiro, a data de processamento e a lista de características om as respetivas traduções e pontos de confiança.

4.4. Consulta de Resultados

A qualquer momento após a submissão, o cliente pode consultar o estado de processamento de uma imagem invocando o método `ObterDetalhesImagem` com o identificador do pedido. O servidor gRPC consulta o Firestore e devolve ao cliente a informação disponível.

Como o processamento é assíncrono, é possível que os resultados ainda não estejam disponíveis no momento da consulta. Nesse caso, o servidor devolve uma resposta com a lista de características vazia, cabendo ao cliente repetir a consulta após um intervalo de tempo.

4.5. Pesquisa por Características

O cliente pode também pesquisar todas as imagens processadas que contenham uma determinada característica num intervalo de datas, invocando o método `ProcurarPorCaracteristica`. O servidor gRPC consulta o Firestore filtrando os documentos pelo nome da característica e pelo intervalo de datas indicando, devovendo ao cliente a lista dos nomes dos ficheiros que satisfazem.

4.6. Logging – Cloud Function Pub/ Sub

De forma opcional, a Cloud Function com *trigger* Pub/ Sub permite registar no Firestore cada pedido de processamento no momento em que é publicado, antes de a *Labels App* o processar. Isto permite manter um registo de todos os pedidos submetidos, independentemente de terem sido ou não processadas com sucesso.

5. Formatos de Dados e Mensagens

Esta secção documenta a estrutura dos dados que circulam entre os componentes do sistema, nomeadamente as mensagens do Pub/ Sub, os documentos do Firestore e os *blobs* no *Cloud Storage*.

5.1. Documentos Firestore

O Firestore em modo *Native* numa base de dados denominada `cn-firestore`, na região `europe-west1`. Os resultados do processamento de cada imagem são guardados na coleção `imagens_processadas`, onde cada documento é identificado pelo identificador único do pedido (UUID). A estrutura de cda documento é a seguinte:

- **`dataProcessamento`** – String no formato `YYY-MM-DD` com a data em que a imagem foi processada pela *Labels App*.
- **`nomeBlob`** – *String* com o nome do *blob* no *Cloud Storage*, no formato `<uuid>_<nome_original_do_ficheiro>`.
- **`labels_simples`** – *Array* de *Strings* com os noems das características detetadas em inglês, utilizado nas pesquisas por característica no Firestore.
- **`Caracterisicas_detalhadas`** – Array de objetos, onde cada objeto contém os campos n`omeOriginal` (*String*), `nomeTraduzido` (*String*) e `confianca` (*Double enter* `0` e `1`).

A separação entre `labels_simples` e `características_detalhadas` é uma decisão de design deliberada: o Firestore suporta eficientemente pesquisas do tipo `whereArrayContains` sobre *arrays* se *Strings* simples, mas não sobre *arrays* de objetos aninhados. Manter uma lista simples em paralelo com a lista detalhada permite fazer pesquisas rápidas por característica sem abdicar da riqueza de informação armazenada. Uma alternativa seria guardar apenas a lista detalhada e filtrar no código do servidor, mas essa abordagem transferiria para a memória da aplicação um trabalho que o Firestore consegue fazer de forma mais eficiente através de índices

Para suportar a pesquisa simultânea por característica e por intervalo de datas, que combina um filtro `whereArrayContains` sobre `labels_simples` com filtros de intervalo sobre `dataProcessamento`, foi necessário criar um índice composto no Firestore. Sem este índice, o Firestore rejeita a `query` por razões de performance. O índice foi criado através de um link gerado automaticamente pelo próprio Firestore no momento em que a `query` foi executada pela primeira vez.

5.2. Mensagem Pub/ Sub

O sistema utiliza um único tópico Pub/Sub para comunicação entre o servidor gRPC e as instâncias da *Labels App*. O formato das mensagens publicadas pelo servidor é uma *String* de texto simples com três campos separados pelo carácter `|`:

`<id_pedido>|<nome_bucket>|<nome_blob>`
Por exemplo: `ff390bc0-2676-4f82-8e58-7374114ff143|cn-trabalho-final-bucket-2526|ff390bc0-..._foto.jpg`

A escolha de um formato de texto simples separado por `|` em vez de JSON ou Protobuf foi uma decisão pragmática: a mensagem contém apenas três campos bem definidos, e um formato simples elimina a dependência de bibliotecas de serialização do lado do consumidor. O risco desta abordagem é a fragilidade face a nomes de ficheiros que contenham o carácter `|`, que foi mitigado pelo facto de os nomes dos blobs serem gerados pelo próprio servidor (com base no UUID) e não diretamente pelo utilizador.

O tópico tem duas subscrições independentes: `cn-labels-App`, utilizada pelas instâncias da *Labels App* segundo o padrão *work-queue*, e `cn-image-processing-sub`, reservada para a *Cloud Function* opcional de *logging*. A existência de duas subscrições independentes garante que cada subscritor recebe uma cópia da mensagem, permitindo que a *Labels App* e a *Cloud Function* de *logging* operem de forma completamente independente.

5.3. Blobs no Cloud Storage

As imagens são armazenadas no *bucket* `cn-trabalho-final-bucket-2526`, configurado na região `europe-west1` com armazenamento detalhado (*detailed*). Cada *blob* é nomeado segundo o formato `<uuid>_<nome_original_do_ficheiro>`, por exemplo `ff390bc0-2676-4f82-8e58-7374114ff143_foto.jpg`. A inclusão do UUID no nome do *blob* evita colisões entre ficheiros com o mesmo nome enviados por utilizadores diferentes, e permite associar diretamente o *blob* ao identificador do pedido sem necessidade de consultar a base de dados.

6. Elasticidade

O sistema foi desenhado para suportar elasticidade independente em duas camadas: os servidores gRPC e as instâncias da *Labels App*.

6.1. Infraestrutura de Suporte

A elasticidade é suportada por dois *Managed Instance Groups* (MIG) distintos na zona `europe-southwest1-a` (Madrid), criados a partir de duas *Instance Templates* diferentes, mas baseadas na mesma imagem de disco (`image-ubuntu-base`):

O grupo `grupo-madrid-servidores` utiliza o *template* `template-madrid-servidor`, cujo *startup script* arranca exclusivamente o servidor gRPC. O grupo `grupo-madrid-workers` utiliza o `template template-madrid-worker`, cujo *startup script* arranca exclusivamente a *Labels App*. Esta separação é fundamental para que os dois grupos possam ser escalados de forma completamente independente.

A decisão de usar dois grupos separados em vez de um único grupo com ambas as aplicações permite, por exemplo, ter 1 servidor gRPC e 5 instâncias da *Labels App* durante um pico de processamento, ou vice-versa. Uma alternativa seria escalar um único grupo com ambas as aplicações, mas tal eliminaria a possibilidade de ajuste independente e desperdiçaria recursos.

O *autoscaling* está desativado em ambos os grupos, pois a gestão do número de instâncias é feita programaticamente pelo próprio sistema através das operações do serviço SG.

6.2. Operações de Escalonamento

As operações `EscalarServidoresGrpc` e `EscalarLabelsApp` do serviço `GestaoElasticidade` comunicam com a API do *Compute Engine* através da classe `InstanceGroupManagersClient`, enviando um pedido de redimensionamento com o número desejado de instâncias. A operação é assíncrona do ponto de vista da *Google Cloud*, mas o servidor aguarda a confirmação da receção do pedido antes de responder ao cliente.

Uma decisão relevante foi expor estas operações de gestão através do mesmo contrato gRPC que as operações funcionais, em vez de criar uma interface de administração separada. Isto simplifica o cliente, que só precisa de conhecer um único ponto de entrada para todas as operações do sistema.

6.3. Descoberta de Serviços

Como o endereço IP das instâncias dos servidores gRPC muda sempre que uma máquina é substituída ou criada, o cliente não pode ter um IP fixo no código. Esta limitação é resolvida através de uma *Cloud Function* HTTP denominada `lookupFunction`, implementada em Java e publicada na região `europe-southwest1`.

Quando o cliente arranca, invoca esta função passando o nome do grupo de instâncias como parâmetro de URL (`?mig=grupo-madrid-servidores`). A função usa a API do *Compute Engine* para listar todas as instâncias em estado `RUNNING` cujo nome começa com o prefixo do grupo indicado, extrai o IP público de cada uma e devolve a lista em formato JSON. O cliente apresenta a lista ao utilizador, que escolhe o servidor ao qual se quer ligar. Se a ligação falhar, o cliente pode pedir uma nova lista ou escolher outro IP.

7. Aspetos Relevantes de Implementação

Esta secção documenta as decisões técnicas mais significativas tomadas durante o desenvolvimento, incluindo problemas encontrados e as soluções adotadas.

7.1. Conflito de Versões do gRPC

O problema mais significativo encontrado durante o desenvolvimento foi um conflito de dependências **Maven** entre módulos. O módulo `grpcContract` foi inicialmente configurado com a versão `1.62.2` do gRPC. No entanto, as bibliotecas do Pub/Sub e do Firestore da *Google Cloud* exigem internamente a versão `1.76.3`, gerando uma incompatibilidade que se manifestava com o erro `NoSuchMethodError: Http2ClientStreamTransportState` no momento em que o servidor tentava autenticar-se com o Pub/Sub.

A solução foi atualizar a versão do gRPC no `pom.xml` do módulo `grpcContract` de `1.62.2` para `1.76.3`, seguida de uma limpeza completa (`clean + install`) para garantir que todos os módulos dependentes foram recompilados com a versão correta. Este problema surgiu também na fase de *deployment*, pois o ficheiro `.jar` enviado para a VM tinha sido gerado antes da atualização, sendo necessário repetir o processo de *clean*, *install* e *package* e substituir o ficheiro na máquina virtual.

7.2. Dependências Transitivas no Cliente

O módulo `grpcClientApp` incluía inicialmente uma dependência desnecessária ao `google-cloud-storage`. Como o cliente não interage diretamente com o *Cloud Storage*, essa responsabilidade pertence ao servidor, esta dependência introduzia bibliotecas da Google que puxavam versões do gRPC incompatíveis com as do contrato, causando o mesmo tipo de conflito descrito na secção anterior. A solução foi remover a dependência do cliente, mantendo apenas a dependência ao `grpcContract`.

7.3. Ordem de Build dos Módulos Maven

A ordem correta de *build* dos módulos **Maven** é a seguinte: primeiro é necessário executar *install* no módulo `grpcContract`, para que o artefacto seja registado no repositório local Maven e fique disponível para os restantes módulos. Só depois se pode executar package nos módulos `grpcServerApp`, `grpcClientApp` e `labelsApp`, sem ordem específica entre eles. Ignorar esta ordem resulta em erros de dependência não resolvida durante a compilação.

7.4. Autenticação nas Máquinas Virtuais

Durante os testes de *deployment*, verificou-se que o comando `export GOOGLE_APPLICATION_CREDENTIALS=key.json` não é preservado quando se usa `sudo` para arrancar o servidor, pois o `sudo` cria um ambiente de execução limpo que descarta as variáveis de ambiente do utilizador. A solução adotada foi passar a variável diretamente na mesma linha do comando de arranque:

```
GOOGLE_APPLICATION_CREDENTIALS=/var/grpcserver/key.json java -jar
/var/grpcserver/server.jar
```

Esta abordagem foi também aplicada nos *startup scripts* dos *templates* de instâncias, garantindo que as aplicações arrancam corretamente de forma autónoma quando uma nova VM é criada pelo grupo de instâncias.

7.5. Esgotamento de Quotas na Região da Bélgica

Durante a criação dos grupos de instâncias, a `zona europe-west1 (Bélgica)` devolveu o erro `ZONE_RESOURCE_POOL_EXHAUSTED`, indicando que o *data center* não tinha capacidade disponível para o tipo de máquina `e2-small` em contas de estudante. A solução foi migrar toda a infraestrutura de *Compute Engine* para a zona `europe-southwest1-a` (Madrid), criando novos *templates* a partir da mesma imagem de disco `image-ubuntu-base`. O *Cloud Storage* e o Firestore mantiveram-se na região `europe-west1` (Bélgica), uma vez que estes serviços geridos não estavam sujeitos às mesmas limitações de quota.

7.6. Project ID na Cloud Function

A *Cloud Function* de lookup usava inicialmente `StorageOptions.getDefaultInstance().getProjectId()` para obter o ID do projeto de forma dinâmica a partir das credenciais. No entanto, no ambiente de execução das *Cloud Functions* de 2ª geração (baseadas em *Cloud Run*), esta variável não é injetada automaticamente, resultando em `null` e causando um erro 500 devolvido ao cliente. A solução foi definir o ID do projeto diretamente no código da função, seguida de um novo *deployment*.

7.7. Publicação Síncrona no Pub/ Sub

A publicação de mensagens no Pub/Sub é por defeito assíncrona na biblioteca Java da Google: o método `publisher.publish()` devolve um `ApiFuture` sem aguardar confirmação. Durante os testes, verificou-se que em caso de falha de autenticação o servidor respondia ao cliente com sucesso sem que a mensagem tivesse sido efetivamente publicada, deixando a imagem presa no *Cloud Storage* sem ser processada. A solução foi forçar a publicação síncrona através de `.get()` sobre o `ApiFuture`, garantindo que o servidor só responde ao cliente após confirmação da receção da mensagem pelo Pub/Sub.

8. Pontos de Falha e Limitações

Esta secção identifica os principais pontos de falha do sistema, a forma como são ou não mitigados, e os objetivos que foram e não foram atingidos.

8.1. Processamento Assíncrono e Indisponibilidade de Resultados

Como o processamento das imagens é assíncrono, existe um intervalo de tempo entre a submissão de uma imagem e a disponibilidade dos seus resultados no Firestore. Durante este intervalo, uma consulta pelo identificador do pedido devolve uma resposta de erro `NOT_FOUND`. O sistema não implementa nenhum mecanismo de notificação proativa ao cliente quando os resultados ficam disponíveis, cabendo ao cliente repetir a consulta manualmente após um intervalo de tempo. Uma alternativa seria implementar *server streaming* no método `ObterDetalhesImagem`, mantendo a ligação aberta até os resultados ficarem disponíveis, mas tal implicaria alterações significativas ao contrato gRPC e à lógica do servidor.

8.2. Tolerância a Falhas no Pub/Sub

O Pub/Sub garante a entrega das mensagens mesmo que as instâncias da *Labels App* estejam temporaneamente indisponíveis, retendo as mensagens na subscrição até que um *worker* as consuma. Este comportamento foi verificado durante os testes: uma imagem submetida enquanto o grupo `grupo-madrid-workers` tinha zero instâncias foi processada corretamente assim que as instâncias foram escaladas para dois. No entanto, o sistema não implementa nenhum mecanismo de *dead letter queue* para mensagens que falhem repetidamente, o que significa que uma mensagem cujo processamento falhe consecutivamente pode ficar indefinidamente retida na subscrição.

8.3. Ponto Único de Falha no Servidor gRPC

Embora o sistema suporte múltiplas instâncias de servidores gRPC através do grupo de instâncias `grupo-madrid-servidores`, o cliente estabelece uma ligação a um único servidor no arranque e mantém essa ligação durante toda a sessão. Se o servidor ao qual o cliente está ligado ficar indisponível durante a sessão, o cliente recebe um erro e tem de reiniciar a aplicação para fazer um novo *lookup* e obter um IP alternativo. Uma mitigação possível seria implementar reconexão automática no cliente, mas tal não foi desenvolvido no âmbito deste projeto.

8.4. Ausência de Autenticação entre Componentes

A comunicação gRPC entre o cliente e o servidor é feita sem TLS e sem autenticação, utilizando o modo `usePlaintext()`. Isto significa que qualquer entidade com acesso à porta 8000 das VMs pode enviar pedidos ao servidor. Esta limitação é aceitável num contexto académico mas seria inaceitável num ambiente de produção, onde se deveria configurar TLS mútuo ou *tokens* de autenticação.

8.5. Nomes Hardcoded na Configuração

Os nomes do *bucket* do *Cloud Storage* e do tópico Pub/Sub estão definidos diretamente no código do servidor (FuncionalidadesGeraisImpl.java), assim como os nomes dos grupos de instâncias e a zona geográfica no servidor (GestaoElasticidadeImpl.java) e o URL da *Cloud Function* no cliente. Embora esta abordagem simplifique o desenvolvimento, qualquer alteração à infraestrutura na consola da *Google Cloud* exige recompilação e novo *deployment* do código. Uma alternativa mais robusta seria usar variáveis de ambiente para todos estes valores.

8.6. Objetivos Atingidos e Não Atingidos

8.6.1. Objetivos Atingidos

- Submissão de imagens em *client streaming* via gRPC com armazenamento no *Cloud Storage*.
- Processamento assíncrono de imagens pela *Labels App* através do Pub/Sub, com deteção de características via *Vision* API e tradução via Translation API.
- Persistência dos resultados no Firestore e consulta pelo cliente através do servidor gRPC.
- Pesquisa de imagens por característica e intervalo de datas, com índice composto no Firestore.
- Elasticidade independente dos servidores gRPC e das instâncias da Labels App através de dois grupos de instâncias geridos.
- Descoberta de serviços dinâmica via *Cloud Function* HTTP, com seleção interativa de IP pelo utilizador e tolerância a falhas de ligação.
- *Startup scripts* nos *templates* de instâncias garantindo arranque automático das aplicações em novas VMs.
- Resiliência da infraestrutura verificada através da destruição e regeneração automática de instâncias pelo grupo de instâncias gerido.

8.6.2. Objetivos Não Atingidos

- A Cloud Function opcional de logging com trigger Pub/Sub para registo de pedidos no Firestore não foi implementada, tendo sido considerada de menor prioridade face aos restantes requisitos.
- A reconexão automática do cliente em caso de falha do servidor durante uma sessão ativa não foi implementada.

9. Conclusões

O sistema CN2026Labels foi desenvolvido com sucesso, integrando de forma coerente os serviços da *Google Cloud Platform* previstos no enunciado. A arquitetura distribuída e assíncrona adotada, assente no desacoplamento entre o servidor gRPC e a *Labels App* através do Pub/Sub, provou ser robusta e elástica, suportando o processamento de imagens de forma independente em cada camada.

Os principais desafios técnicos encontrados foram conflitos de versões do gRPC, gestão de autenticação nas VMs, esgotamento de quotas regionais e comportamento da *Cloud Function* de 2ª geração. Estes, foram todos identificados e resolvidos, contribuindo para uma compreensão mais aprofundada das especificidades e limitações das plataformas de computação na nuvem em contexto real.